Polymorphism

Abstract Classes, Abstract Methods, Override Methods

SoftUni Team

Technical Trainers

Software University

SoftUni Foundation

Software University

http://softuni.bg

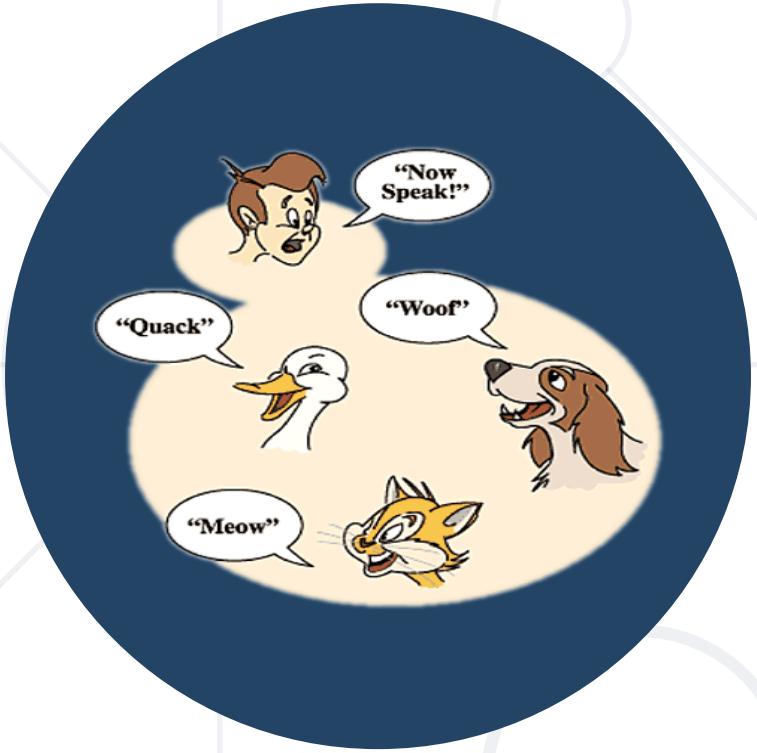
Table of Contents

Questions

SoftUni Foundation

sli.do

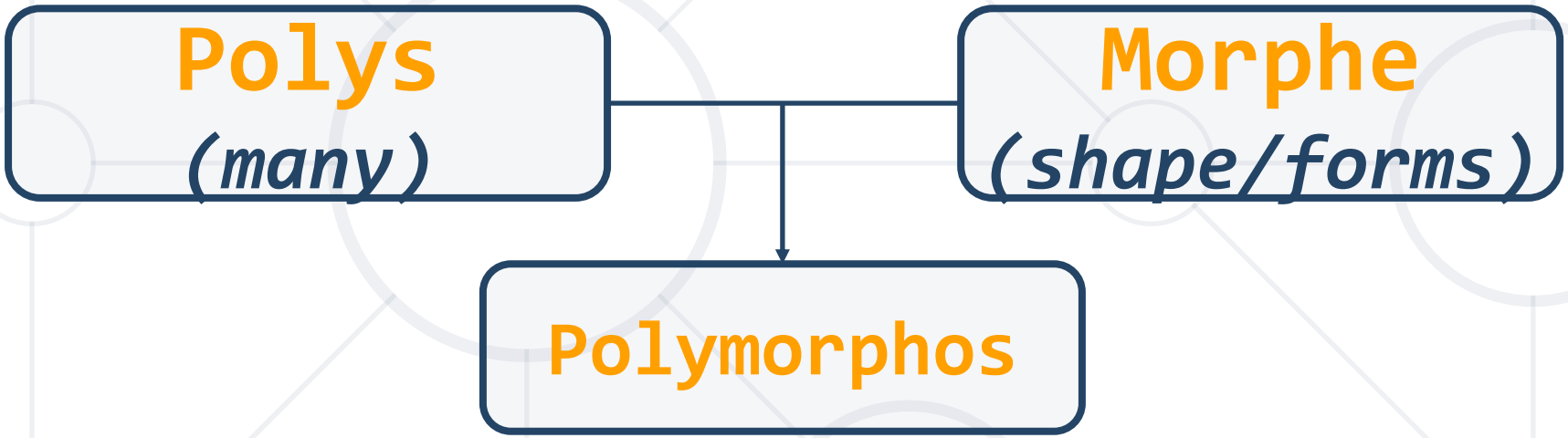
#fund-csharp



Polymorphism

What is Polimorphism?

- From the Greek



- This is something similar to a word having several different meanings depending on the context
- Polymorphism is often referred to as the third pillar of object-oriented programming, after encapsulation and inheritance

Polymorphism in OOP

SoftUni Foundation

- Ability of an **object** to take on **many forms**

```
public interface IAnimal {}
public abstract class Mammal {}
public class Person : Mammal, IAnimal {}
```

```
Person IS-A Person
```

```
Person IS-AN Animal
```

```
Person IS-A Mammal
```

```
Person IS-AN Object
```

Reference Type and Object Type

- **Variables** are saved in a **reference** type
- You can use only **reference methods**
- If you need an **object method** you need to **cast it or override it**

```
public class Person : Mammal, Animal {}
Animal person    = new Person();
Mammal personOne = new Person();
Person personTwo = new Person();
```

Reference Type

Object Type

Keyword - is

- Check if an **object** is an **instance** of a specific **class**

```
public class Person : Mammal, Animal {}
Animal person = new Person();
Mammal personOne = new Person();
Person personTwo = new Person();
if (person is Person)
{
  ((Person) person).getSalary();
}
```

Check object type of person

Cast to object type and use its methods

Keyword - is

- Starting with C# 7.0, the **is** statement supports **pattern matching**. The is keyword supports the following patterns:
  - **Type pattern**, which tests whether an expression can be converted to a specified type and, if it can be, casts it to a variable of that type
  - **Constant pattern**, which tests whether an expression evaluates to a specified constant value
  - **var pattern**, a match that always succeeds and binds the value of an expression to a new local variable

Type pattern

SoftUni Foundation

```
public class Person : Mammal, Animal {}
Mammal personOne = new Person();
Person personTwo = new Person();
if (personTwo is Person person)
{
  person.getSalary();
}
```

Checks if object is of type person and casts it

Uses its methods

Constant pattern

- When performing pattern matching with the constant pattern, **is** tests whether an expression equals a specified constant
- Checking for **null** can be performed using the constant pattern

```
int i = 0;
const max = 10;
while(true)
{
  Console.WriteLine($"i is {i}");
  i++;
  if(i is max) break;
}
```

var pattern

- A pattern match with the **var pattern** always succeeds

```
If(expr is var varname)
{
  // Do something with varname
}
```

- The value of **expr** is always assigned to a local variable named **varname**
- **varname** is a static variable of the same type as **expr**
- Note that if **expr** is null, the is expression still is true and assigns null to **varname**

Keyword - is

SoftUni Foundation

```
Anytime you find yourself writing code of the
form "if the object is of type T1, then do
something, but if it's of type T2, then do
something else", slap yourself.
```

From *Effective C++*, by Scott Meyers

Keyword - as

- You can use the **as** operator to perform certain types of conversions between compatible reference types

```
public class Person : Mammal, Animal {}
Animal person = new Person();
Mammal personOne = new Person();
Person personTwo;
personTwo = personOne as Person;
if (personTwo != null)
{
    // Do something specific for Person
}
```

Convert Mammal to Person

Check if conversion is successful

Types of Polymorphism

- Runtime

```
public class Shape {}
public class Circle : Shape {}
public static void Main()
{
   Shape shape = new Circle()
}
```

- Compile time

```
public static void Main()
{
   int Sum(int a, int b, int c)
   double Sum(Double a, Double b)
}
```

Compile Time Polymorphism

SoftUni Foundation

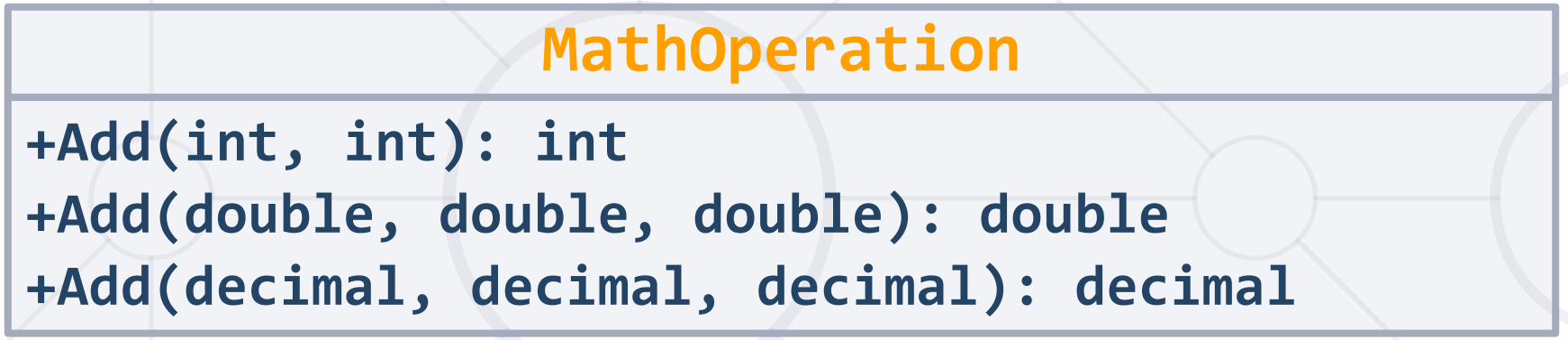
- Also known as **Static Polymorphism**

```
public static void Main()
{
  static int MyMethod(int a, int b) {}
  static double MyMethod(double a, double b) { … }
}
```

Method overloading

- Argument lists could differ in:
  - Number of parameters
  - Data type of parameters
  - Order of parameters

Problem: MathOperation



```
MathOperations mo = new MathOperations();
Console.WriteLine(mo.Add(2, 3));
Console.WriteLine(mo.Add(2.2, 3.3, 5.5));
Console.WriteLine(mo.Add(2.2m, 3.3m, 4.4m));
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: MathOperation

```
public int Add(int a, int b)
{
  return a + b;
}
public double Add(double a, double b, double c)
{
  return a + b + c;
}
public decimal Add(decimal a, decimal b, decimal c)
{
  return a + b + c;
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Rules for Overloading a Method

- Signature **should be different**
  - **Number** of arguments
  - **Type** of arguments
  - **Order** of arguments
- Return type is not a part of its signature
- Overloading can take place in the **same class** or in its **sub-classes**
- Constructors can be **overloaded**

Runtime Polymorphism

SoftUni Foundation

- Has two distinct aspects:
- At run time, objects of a derived class may be treated as objects of a base class in places such as method parameters and collections or arrays. When this occurs, the object's declared type is no longer identical to its run-time type
- Base classes may define and implement virtual methods, and derived classes can override them, which means they provide their own definition and implementation. At run-time, when client code calls the method, the CLR looks up the run-time type of the object, and invokes that override of the virtual method. Thus in your source code you can call a method on a base class, and cause a derived class's version of the method to be executed.

Runtime Polymorphism

- Also known as **Dynamic Polymorphism**

```
public class Rectangle {
  public virtual double Area() {
    return this.a * this.b;
  }
}
public class Square : Rectangle {
  public override double Area() {
    return this.a * this.a;
  }
}
```

Method overriding

Runtime Polymorphism (2)

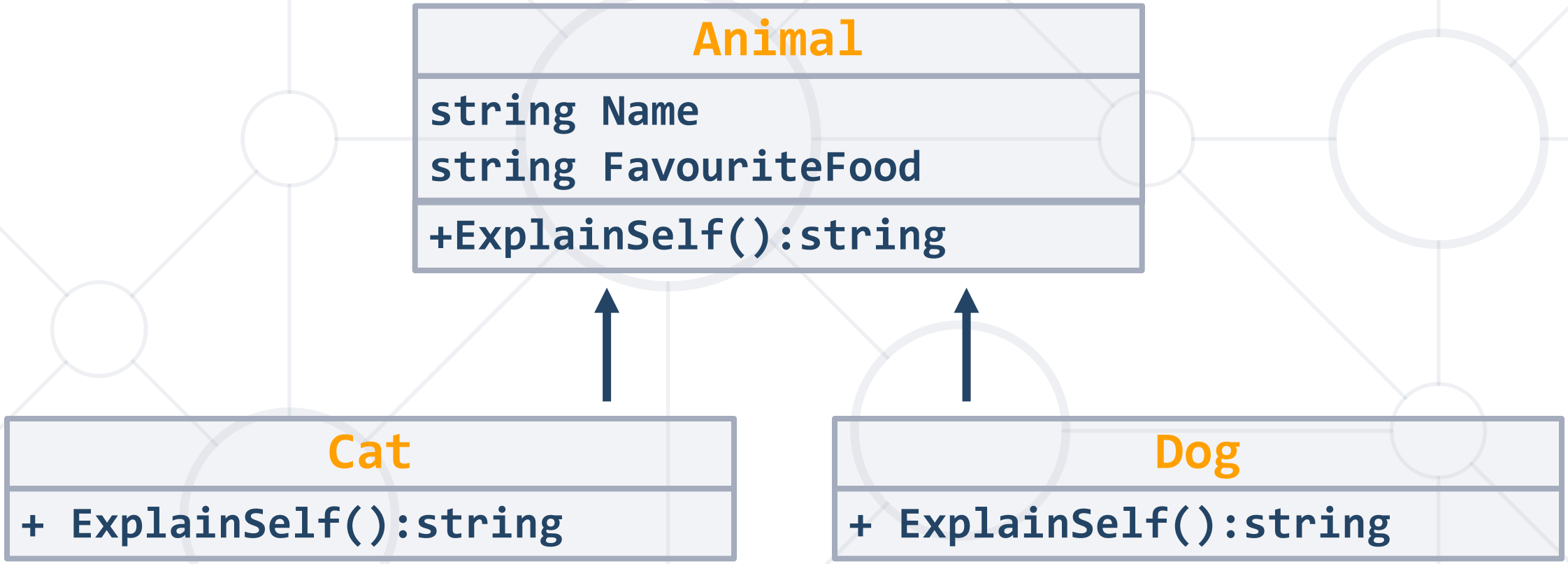
- Usage of **override** method

```
public static void Main()
{
  Rectangle rect = new Rectangle(3.0, 4.0);
  Rectangle square = new Square(4.0);

  Console.WriteLine(rect.Area());
  Console.WriteLine(square.Area());
}
```

Method overriding

Problem: Animals



Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: Animals

```
public abstract class Animal
{
  public string Name { get; protected set; }
  public string FavouriteFood { get; private set; }
  public virtual string ExplainSelf() {
    return string.Format(
      "I am {0} and my favourite food is {1}",
      this.Name,
      this.FavouriteFood";
  }
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: Animals (2)

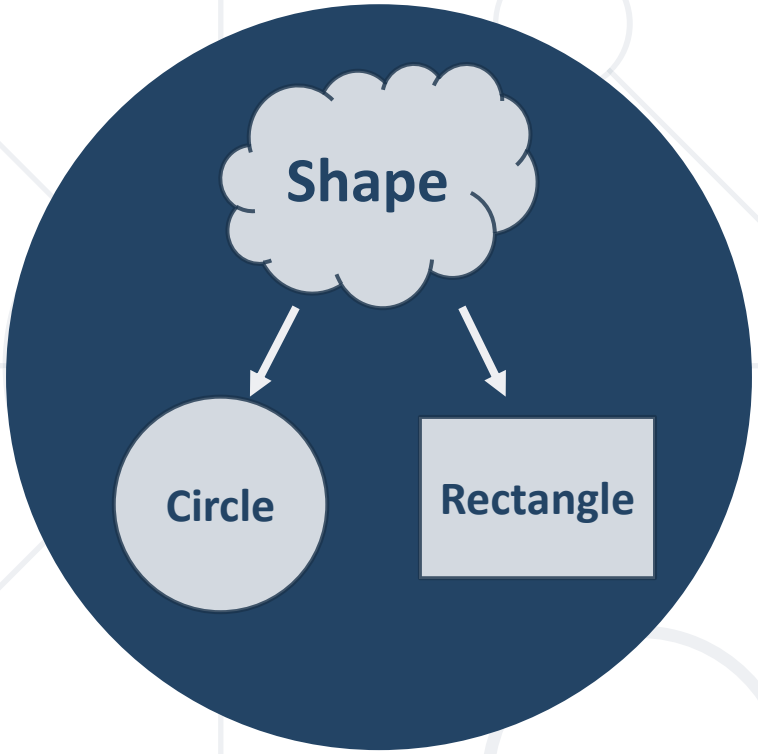
```
public class Dog : Animal
{
  public Dog(string name, string favouriteFood)
    : base(name, favouriteFood) { }
  public override string ExplainSelf()
  {
    return base.ExplainSelf() + Environment.NewLine +
                                          "DJAAF";
  }
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Rules for Overriding Method

SoftUni Foundation

- **Overriding** must take place in any sub-classes
- The overriding method and the base must have the **same return type** and the **same signature**
- Base method must have the **virtual** keyword
- Overriding method must have the **abstract** or **override** keyword
- **Private and static** methods **cannot** be overridden



Abstract Classes

Abstract Classes

SoftUni Foundation

- Abstract class **can NOT be instantiated**

```
public abstract class Shape {}
public class Circle : Shape {}
Shape shape = new Shape(); // Compile time error
Shape circle = new Circle(); // Polymorphism
```

- An **abstract** class may or may not include **abstract methods**.
- If it has at least one abstract method, it must be declared **abstract**
- To use an **abstract class**, you need to **inherit it**

Abstract Classes Elements

SoftUni Foundation

```
public abstract class Shape
{
    private Point startPoint;
    protected Shape(Point startPoint)
    {
        this.startPoint = startPoint;
    }
    public Point StartPoint() => this.startPoint;
    public abstract void Draw();
}
```

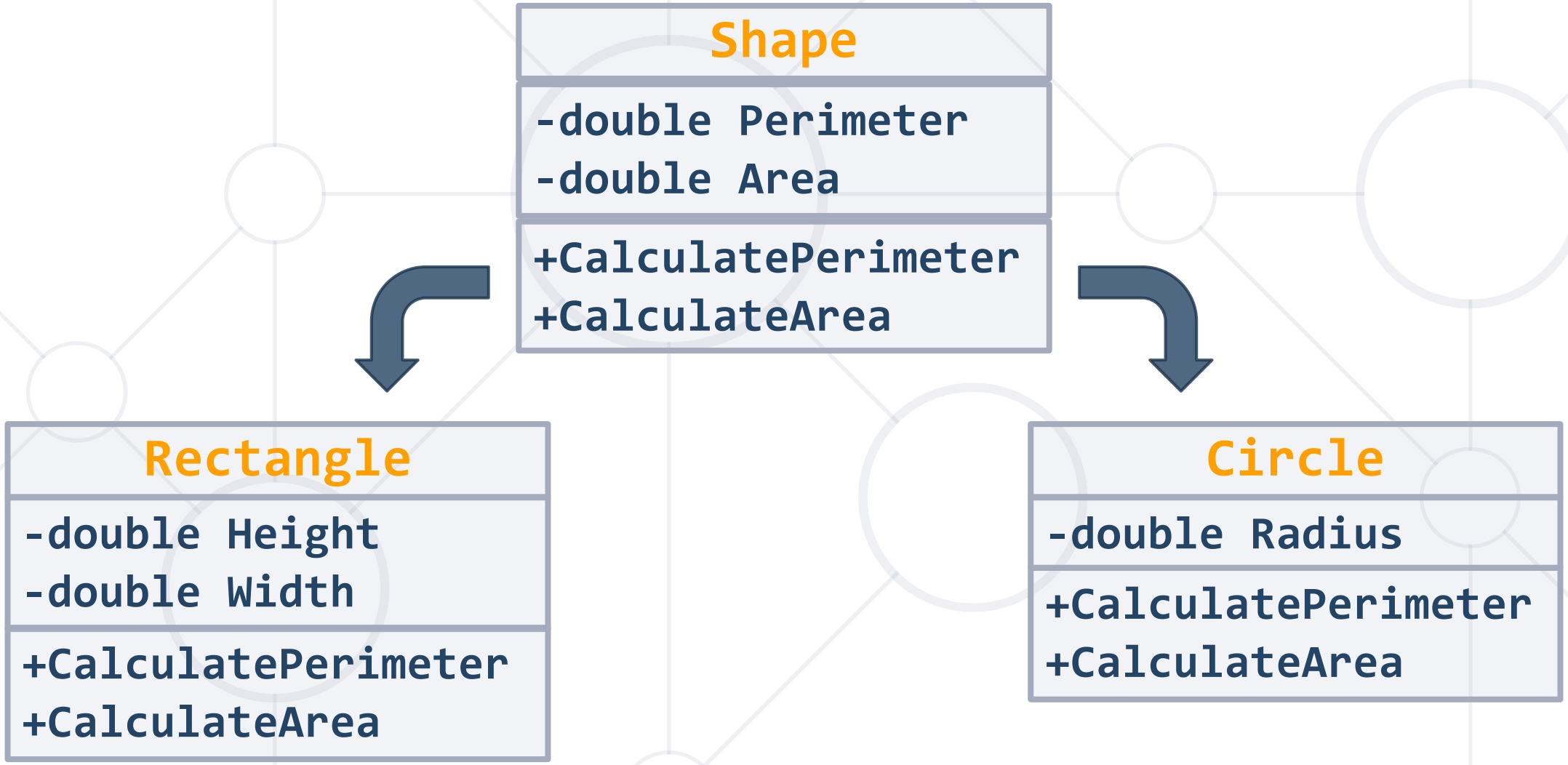
Can have fields

Can have constructor

Can have methods

Implemented by child classes

Problem: Shapes



Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: Shapes

SoftUni Foundation

```
public abstract class Shape
{
    public abstract double CalculatePerimeter();

    public abstract double CalculateArea();

    public virtual string Draw()
    {
        return "Drawing ";
    }
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: Shapes (2)

SoftUni Foundation

```
public class Rectangle : Shape
{
  //TODO: Add fields and constructor
  public override double CalculatePerimeter()
  { return this.sideA * 2 + this.sideB * 2; }

  public override double CalculateArea()

  { return this.sideA * this.sideB; }

  public sealed override string Draw()

  { return base.Draw() + "Rectangle"; }
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Solution: Shapes (3)

SoftUni Foundation

```
public class Circle : Shape
{
  //TODO: Add fields and constructor
  public override double CalculatePerimeter()
  { return 2 * Math.PI * this.radius; }

  public override double CalculateArea()

  { return Math.PI * this.radius * this.radius; }

  public sealed override string Draw()

  { return base.Draw() + "Circle"; }
}
```

Check your solution here: https://judge.softuni.bg/Contests/680/Polymorphism-Lab

Keyword - sealed

SoftUni Foundation

- Modifier **prevents** other classes from **inheriting** from it

```
public abstract class Shape {}
public sealed class Rectangle : Shape {}
public class Sqaure : Rectangle {}        // Compile time error
```

```
public class Rectangle : Shape {
  public sealed override double GetArea() { … }
}
public class Square : Rectangle {
  public override double GetArea() { … } // Compile time error
}
```

- **Allows** classes **to derive from your class** and **prevent** them from overriding specific **virtual methods** or properties

Summary

- Polymorphism - **Definition** and **Types**
- Override Methods
- Overload Methods
- Abstraction
  - **Classes**
  - **Methods**

SoftUni Diamond Partners

SoftUni Foundation

XSsoftware

SoftwareGroup
doing it right

NETPEAK

SmartIT

SUPER HOSTING .BG

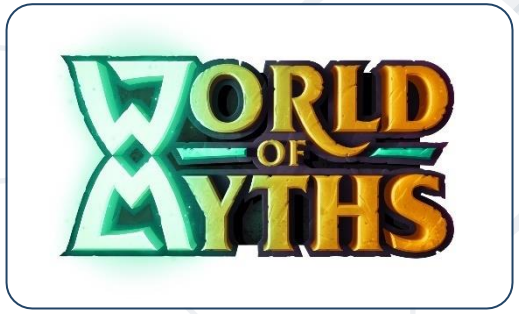
INDEAVR
Serving the high achievers

INFRAGISTICS®

LIEBHERR

SoftUni Organizational Partners

SoftUni Foundation



Trainings @ Software University (SoftUni)

SoftUni Foundation

- Software University – High-Quality Education and Employment Opportunities
  - softuni.bg
- Software University Foundation
  - http://softuni.foundation/
- Software University @ Facebook
  - facebook.com/SoftwareUniversity
- Software University Forums
  - forum.softuni.bg

License

- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license